Presentation Script

What to show, what to type and what to say, step by step

Mihir Bhati (24BCE5341) · Ahana Sharan (24BCE5334)

About 20 minutes, plus questions

How to use this script. Each step has: **show** (what is on screen), **type** (the command, if any), **say** (lines you can speak; use your own words once you know them) and **if asked** (a likely question). Steps marked *optional* can be skipped if time is short. Rehearse the whole thing once before the real demo.

Before the demo (10 minutes earlier)

☐ Laptop plugged in; Wi-Fi not needed (everything runs offline).
☐ Open the **Ollama** app from the Start menu (it runs the AI model).
☐ Open **PowerShell** and type `cd D:\MyClaudeProject\nlp`.
☐ Type `ollama list` and check `qwen2.5:3b` is listed.
☐ Run `python scripts/demo.py --project P1` once, so the model is warmed up (the first answer is slow).
☐ Open in File Explorer: `reports\figures` (the charts) and `reports\guide` (the documents).
☐ Keep **How Our System Works.pdf** open in a second window as your backup notes.

If any command fails during the demo, it is almost always because Ollama is closed. Open it, wait 10 seconds and run the command again. Say: "The AI model runs locally in Ollama; let me start it."

The flow at a glance

#	Step	Time	What it proves
1	The problem and the idea	2 min	you understand what you built
2	How the system works	3 min	the agent, the 12 designs, qwen's role
3	Live demo of the finished pipeline	2 min	it works
4	One test, step by step (walkthrough)	3 min	how every test is done
5	The agent choosing a design	2 min	the agentic part
6	The results and the comparisons	4 min	it is better, and where it is not
7	Proof the numbers are real	2 min	results can be checked independently
8	Built it yourself (optional)	2 min	you understand it well enough to rebuild it
9	Honest limits and next step	1 min	maturity and honesty
10	Close, then questions	1 min	

Step 1. The problem and the idea (2 min)

Show: the first page of How Our System Works.pdf, or just talk.

Say:

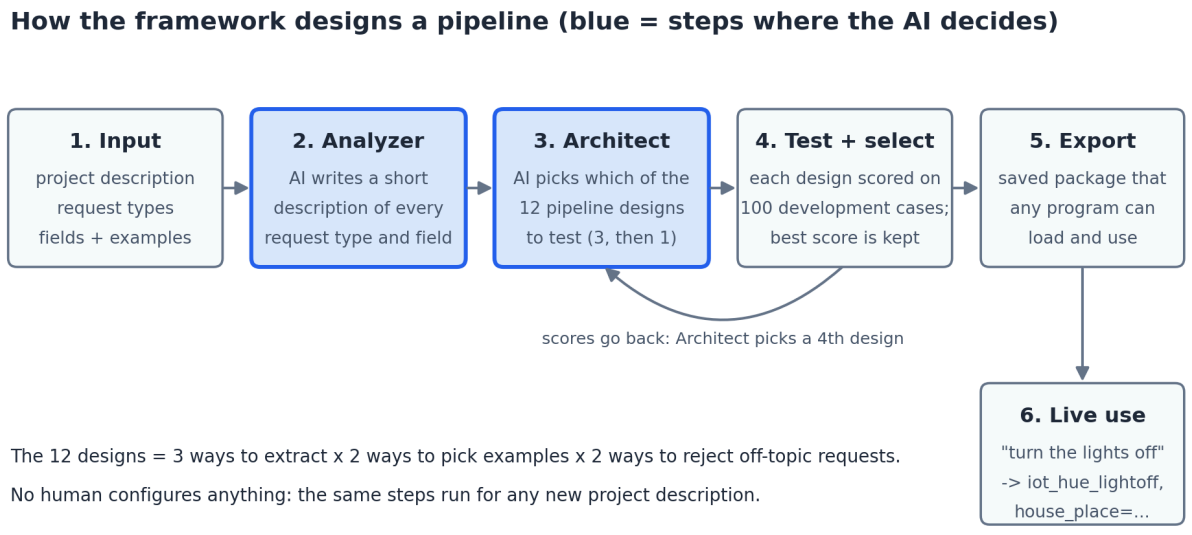
> "Every app that understands users, like a home assistant or a hospital bot, needs an NLP pipeline. It turns a sentence like 'turn off the bedroom lights' into something a program can act on: the intent, lights off, and the detail, bedroom."
> "There are two ways to build that. You can train a model on examples, like XLM-R, the model the MASSIVE dataset paper used. Or you can give a ready-made AI, like qwen, a well-written prompt."
> "Writing that prompt well takes trial and error for every new app. Our framework automates it: from a description of the app, an AI agent designs the pipeline, tests it and delivers the best one."

If asked:

Question	Answer
What is an intent?	What the user wants to do. 'Turn off the lights' has the intent iot_hue_lightoff.
What is NLP?	Natural language processing: getting computers to work with human language.

Step 2. How the system works (3 min)

Show: the diagram below (also in How Our System Works, section 9).



The framework's steps. Blue = where the AI decides.

Say:

> "You give the framework a description of the app, its list of intents and about 10 example sentences per intent."
> "First the Analyzer: the AI writes a short description of every intent."
> "Then the Architect: there are 12 ways to write the prompt, which we call designs. The AI picks 3 it thinks will work, our program tests each on 100 practice sentences, the AI sees the scores and picks a 4th."
> "The best of the 4 is kept and saved as a package any app can load."
> "Important: qwen itself never changes. We are not training it. We only choose the best way to ask it."

If asked:

Question	Answer
Is this training?	No. Training changes the model, like XLM-R. Here the model stays the same; we only choose the best prompt.
Who makes the pipeline, qwen?	No. Our Python program builds and tests it. qwen gives advice on which designs to try, and later answers each request.
What are the 12 designs?	3 ways to ask × 2 ways to choose examples × 2 ways to refuse off-topic requests.

Step 3. Live demo of the finished pipeline (2 min)

Show: PowerShell.

Type:

```
python scripts/demo.py --project P1
```

Say:

> "This is the pipeline our agent chose for the home-assistant project, loaded fresh as a saved package."
> "It handles normal requests, refuses off-topic ones, and rejects invalid input before even calling the AI."

Point at: '-> ok intent=alarm_set time=six thirty' (understood, with the detail extracted); '-> out_of_scope' (correctly refused); '-> invalid_request' (empty input refused).

If asked:

Question	Answer
Can I try my own sentence?	Yes: run python scripts/demo.py --project P1 --interactive and let the teacher type. Press Enter on an empty line to stop.
Does it need the internet?	No. The AI model runs locally on this laptop.

Step 4. One test, step by step (3 min)

Show: PowerShell.

Type:

```
python scripts/walkthrough.py
```

Say:

> "This shows exactly how every test works, for three requests."
> "Step 1 is the correct answer, from the dataset, labelled by people. Step 3 is the prompt we send to the AI. Step 4 is its raw reply. Step 5 is how we read and check it. Step 6 is whether it was right."
> "A real experiment does exactly this for all 100 practice requests, saves every answer, and computes the F1 score."

If asked:

Question	Answer
Where does the correct answer come from?	data\processed\P1\dev.jsonl: the MASSIVE dataset, labelled by people.
How is F1 computed?	Run python scripts/explain_f1.py: it puts the correct answers and the system's answers side by side and shows every count and the formula. Ours gives 0.974, the same as the recorded result and scikit-learn.

Step 5. The agent choosing a design (2 min)

Show: PowerShell.

Type:

```
python scripts/run_a2.py
```

Say:

> "Here the agent designs the pipeline itself. First the AI ranks each design choice and gives its reasons."
> "Our program builds 3 designs from that, tests them, shows the AI the scores, and the AI changes one thing."
> "Notice the AI's first preferences were partly wrong, but testing caught that: the 4th design scored 0.974, the best of all 12 designs, while testing only 4."

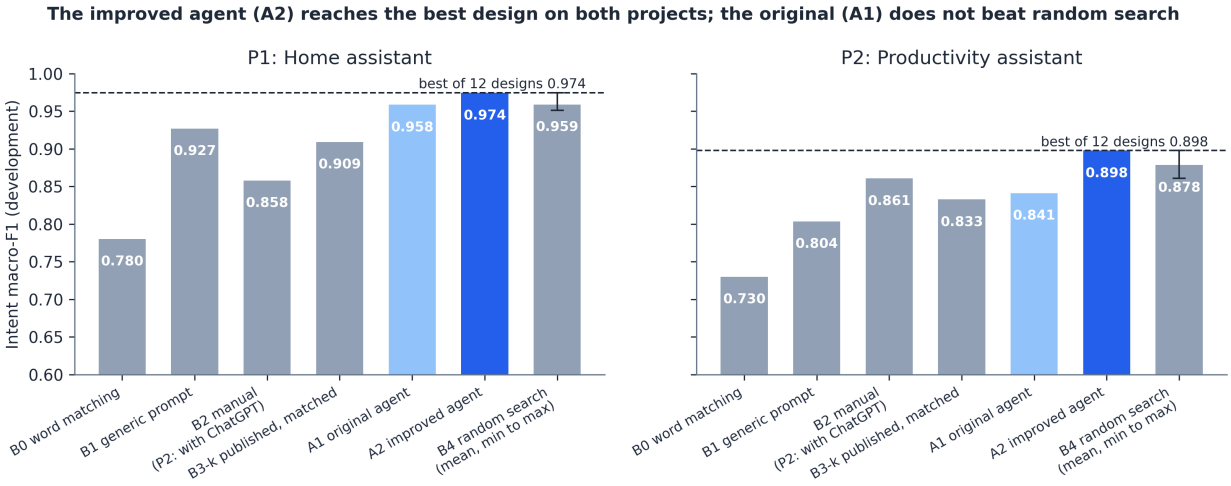
Point at: the 'ranked' lines (the AI's reasoning), the 'tried' lines with their scores, and 'selected schema-sim-llm: intent 0.974'.

If asked:

Question	Answer
What makes it agentic?	The AI makes design decisions, sees the results of its own choices, and revises them, with no human configuring anything.

Step 6. The results and the comparisons (4 min)

Show: reports\figures\fig1_main_comparison.png, then fig2_intent_vs_entity.png.



Intent scores for every system on both projects.

Say:

"To prove it is better, we compared it with other ways of building the pipeline, all on the same requests, with the same AI model and the same scorer."
"B0 is simple word matching. B1 is one standard prompt for every app. B2 is a person doing it by hand. B3 is XLM-R trained on the same examples, the published baseline. B4 picks designs at random."
"Our improved agent, A2, scored 0.974 and 0.898: the best of all 12 designs on both projects. It beats the generic prompt, the person working alone, the published baseline, and random search."
"We also report where it was weaker: our first version, A1, did not beat random search, and that is why we built A2."

If asked:

Question	Answer
Why compare with random search?	To check the agent's choices are smart, not lucky. A1 was no better than random; A2 is.
What is XLM-R?	A language model that has to be trained on examples. With only 10 examples per intent it scored 0.909; ours scored 0.974 without any training.
Are these final results?	These are development scores. The final ones come from 300 locked test requests per project, after we freeze the settings.

Step 7. Proof the numbers are real (2 min)

Show: PowerShell.

Type:

```
python scripts/verify_with_sklearn.py
```

Say:

> "Every answer the systems gave is saved, so any result can be recomputed."
> "This script recomputes our scores with scikit-learn, the standard library, without using our own scoring code. Every run agrees, to 9 decimal places."

Point at: every line says 'agrees'. For everything at once: python scripts/show_evidence.py (ends with 'Overall: all checks passed').

If asked:

Question	Answer
How do I know you did not edit the numbers?	reproduce_metrics.py re-scores every run from its saved answers, and a test proves it catches a hand-edited number.
Did you tune on the test data?	No. The code locks the test data until the freeze, and a test checks it.

Step 8. Built it yourself (optional, 2 min)

Show: diy\c04_b1.py open in VS Code, then PowerShell.

Type:

```
python diy/c04_b1.py
```

Say:

> "To make sure we understand it, we also wrote small versions of each test from scratch."
> "This 40-line script builds a prompt, asks the AI about every request and scores it. It gets 0.927, the same as the project's B1 baseline."

If asked:

Question	Answer
Can you change something?	Yes: in diy\c03_b0.py change 0.35 to 0.5 and run it again; the score changes because more requests are refused.

Step 9. Honest limits and next step (1 min)

Show: How Our System Works, sections 14 and 15.

Say:

> "The final pipeline is always one of the 12 designs, customised for the project. That keeps it safe and lets us prove whether the agent is smart."
> "Limits: one small model, development data so far, and the human baseline is one beginner per project."
> "Next, we plan A3: the agent will start from the best design and rewrite its own prompt from its mistakes, keeping only changes that improve the score."

If asked:

Question	Answer
Why not let it design anything?	Then generated code would run, we could not compare it fairly, and our small 3B model already makes mistakes. Prompt refinement (A3) keeps it safe and measurable.

Step 10. Close (1 min)

Show: nothing; look at the teacher.

Say:

> "To sum up: our framework automates designing an NLP pipeline for any new app. It is tested against a generic prompt, a person, random search and the published baseline, and the results can be recomputed by anyone."
> "We used Claude Code, an AI coding assistant, to write the code and run the experiments; we planned it, checked the results, and can explain and re-run every step. That is recorded in docs\contributions.md."
> "Thank you. We are happy to answer questions or run anything again."

Quick answers for questions

If the teacher asks...	Say	Show
What did you build?	A framework that automatically designs, tests and delivers an NLP pipeline for any new app.	How Our System Works, section 1
How is it different from XLM-R?	XLM-R learns by training; our framework designs the best prompt for a model that never changes.	section 4 diagram
What does qwen do?	Gives advice while designing, and answers each request once the pipeline runs.	section 6
What is in the prompt?	The intent list, short descriptions, real example sentences and the request.	section 7
How was each test done?	Same 100 requests, same model, same scorer; every answer saved.	python scripts/walkthrough.py
Show me F1 by hand.	Correct vs system answers, counted per intent, averaged.	python scripts/explain_f1.py
Is the difference real?	Not yet certain on 100 requests; the test set will tell.	python scripts/explain_stats.py bootstrap
How fast is it?	Under one second per request.	python scripts/explain_stats.py timing
What mistakes does it make?	Mostly extracting extra words as details.	python scripts/explain_stats.py errors
Run all your tests.	371 automated tests.	python -m pytest tests
Is it innovative?	It automates the whole design and is rigorously compared; A2 shows structured search matters. Next: A3 refinement.	section 14
Did you use AI?	Yes, Claude Code wrote the code; we planned, checked and can explain everything. It is recorded.	docs\contributions.md

Backup if the live demo fails

- Open the documents instead: **How_Every_Test_Was_Done_v2.pdf** shows the real output of every command.
- Open **reports\EVIDENCE_SUMMARY.md**: the saved output of all checks.
- Charts are in **reports\figures**; they do not need the model.